IIS Installation

This guide installs or updates a published Gestiona Gateway API package as an IIS website using `Install-IIS.ps1`.

Requirements

- A supported Windows or Windows Server installation with IIS enabled.
- The IIS Management Scripts and Tools feature, which provides the PowerShell `WebAdministration` module.
- The .NET 8 Hosting Bundle, including ASP.NET Core Module V2. Restart IIS after installing the bundle.
- Windows PowerShell running as Administrator.
- A ZIP containing the published application. It must contain exactly one `GestionaGatewayAPI.dll` and a `web.config` in the published application directory.
- An unused HTTP port, a unique IIS site name, and an installation directory that does not exist or is empty.
- Network access from the server to the configured Gestiona API endpoint.

The .NET SDK is required only when producing the package on the server. It is not required to run an application deployed with the .NET 8 Hosting Bundle.

Install

Open Windows PowerShell as Administrator, change to the directory containing the installer, and run:

```
.\Install-IIS.ps1 `
    -ZipPath C:\publish\GestionaGatewayAPI.zip `
    -Port 8080 `
    -InstallPath C:\inetpub\GestionaGatewayAPI `
    -SiteName GestionaGatewayAPI
```

`SiteName` is optional and defaults to `GestionaGatewayAPI`.

Parameter	Required	Description
`ZipPath`	Yes	Path to the published application ZIP.
`Operation`	No	`Install` (the default) or `Update`.
`Port`	Install only	HTTP port from 1 through 65535. It must not already be used by an IIS binding.
`InstallPath`	Install only	Destination for the application files. It must be empty if it already exists and cannot be a drive root.
`SiteName`	No	IIS website name. Defaults to `GestionaGatewayAPI`; a new application pool uses the same name.

The script performs the following operations:

1. Validates the ZIP, IIS module, ASP.NET Core Module V2, port, site name, and destination.
2. Extracts and copies the published application.
3. Creates a `logs` directory.
4. Creates an IIS application pool using `ApplicationPoolIdentity` and no managed CLR.
5. Grants the application pool read access to the application and modify access to `logs`.
6. Creates and starts an HTTP website bound to all local IP addresses on the requested port.

If a new installation fails, the script removes the website, application pool, and files created by that installation attempt. It refuses to overwrite an unrelated website, application pool, IIS port binding, or non-empty installation path.

To install another instance alongside the first, use a different site name, port, and installation path:

```
.\Install-IIS.ps1 `
    -ZipPath C:\publish\GestionaGatewayAPI.zip `
    -Port 8081 `
    -InstallPath C:\inetpub\GestionaGatewayAPI-Test `
    -SiteName GestionaGatewayAPI-Test
```

Update

Select the update operation and provide the new package. Specify `SiteName` only when the existing site does not use the default name:

```
.\Install-IIS.ps1 `
    -Operation Update `
    -ZipPath C:\publish\GestionaGatewayAPI.zip `
    -SiteName GestionaGatewayAPI
```

The installer obtains the physical path, assigned application pool, and HTTP port from the existing IIS website. An update is allowed only when the named website, its assigned application pool, physical path, and an HTTP binding all exist.

Before replacing files, the installer validates the new package and copies the current application to a temporary backup. During the update it:

1. Records whether the website and application pool are running.
2. Stops the running website and application pool.
3. Replaces the deployed application files.
4. Preserves the existing `logs` directory and root-level `appsettings*.json` files, as well as `web.config`.
5. Restores the previous running or stopped state.

If replacement fails, the previous application files and service state are restored automatically. The temporary package and backup directories are removed after completion.

Configuration

Configure the deployed `appsettings.json` before starting production traffic, or provide equivalent environment variables. ASP.NET Core environment variable names use a double underscore (`__`) in place of the configuration colon (`:`).

Setting	Environment variable	Description
`Gestiona:GestionaApiBaseUrl`	`Gestiona__GestionaApiBaseUrl`	Base URL of the Gestiona REST API.
`Gestiona:AccessToken`	`Gestiona__AccessToken`	Default Gestiona access token. This value must be defined as a system environment variable and must not be stored in `appsettings.json`. Requests that support `X-User-Access-Token` use that header in preference to this value.
`Gestiona:AddonToken`	`Gestiona__AddonToken`	Token used for add-on authorization operations.
`Serilog:MinimumLevel:Default`	`Serilog__MinimumLevel__Default`	Application log threshold, such as `Information` or `Debug`.

Example configuration:

```
{
  "Gestiona": {
    "GestionaApiBaseUrl": "https://gestiona.example/rest",
    "AddonToken": "replace-with-a-secret"
  }
}
```

Define the access token as a machine-level Windows environment variable from an elevated PowerShell session:

```
[Environment]::SetEnvironmentVariable(
    "Gestiona__AccessToken",
    "replace-with-a-secret",
    [EnvironmentVariableTarget]::Machine)
```

Do not add `Gestiona:AccessToken` to `appsettings.json` or commit production tokens. After creating or changing the system environment variable, restart IIS so its worker processes receive the new value:

```
iisreset
```

After changing other application configuration, recycle the application pool:

```
Restart-WebAppPool -Name GestionaGatewayAPI
```

Serilog writes rolling files under `<InstallPath>\logs`. Setting the minimum level to `Debug` may log request details and should be enabled only when needed.

Verify the Installation

Check the IIS site and application pool:

```
Get-Website -Name GestionaGatewayAPI
Get-WebAppPoolState -Name GestionaGatewayAPI
```

The installer prints the local binding when it completes, for example:

```
http://localhost:8080
```

Review `<InstallPath>\logs` if the application does not start. Swagger UI is enabled only when the application runs in the `Development` environment.

The installer creates an HTTP binding only. Configure the server certificate and an IIS HTTPS binding separately when TLS is required.